

Low-Level Design (LLD): RAG-Based Customer Support Assistant

1. Module-Level Design

- Document Processing Module: Uses [PyPDFLoader] to parse PDFs into text objects.
- Chunking Module: Utilizes [RecursiveCharacterTextSplitter] with [chunk_size=1000] and [chunk_overlap=200] to preserve boundary context.
- Embedding Module: Implements [OpenAIEmbeddings] (or [HuggingFaceEmbeddings]) to convert chunks into 1536-dimensional float arrays.
- Vector Storage Module: Initializes [Chroma] client. Contains methods for [add_documents] and [similarity_search].
- Retrieval Module: Acts as a wrapper around Chroma's retriever interface ([as_retriever(search_kwargs={"k": 3})]).
- Query Processing Module: LLM prompt template that takes [{context}] and [{question}] to generate the final string output.
- Graph Execution Module: Defines a [StateGraph] using LangGraph. Compiles the graph and exposes an [invoke] method.
- HITL Module: A specialized node in the graph that uses LangGraph's [interrupt] feature or simply returns a structured response prompting a human to take over the [State].

2. Data Structures

- Document Representation: LangChain [Document] object: [{"page_content": str, "metadata": {"source": str, "page": int}}].
- Chunk Format: Same as Document Representation, but [page_content] is limited to the chunk size constraint.
- Embedding Structure: List of floats [[0.012, -0.045, ...]].
- Query-Response Schema:

[See diagram]

- State Object for Graph: Uses the [QueryState] dictionary to pass data continuously between nodes.

3. Workflow Design (LangGraph)

- Nodes:
 - [retrieve_and_generate_node(state)]: Connects to the Vector Store, retrieves chunks, queries the LLM, and updates [state["answer"]].
 - [hitl_escalation_node(state)]: Flags the conversation for human intervention and updates [state["answer"]] with an escalation message.
- Edges:
 - [START -> router_edge]
 - [router_edge -> retrieve_and_generate_node] (Condition: standard query)
 - [router_edge -> hitl_escalation_node] (Condition: requires human)
 - [retrieve_and_generate_node -> END]
 - [hitl_escalation_node -> END]

- State: The [QueryState] flows through these edges. Each node returns a dictionary updating specific keys in the state.

4. Conditional Routing Logic

A lightweight LLM call or rule-based router evaluates the [state["question"]] before main processing.

- Escalation criteria:
 - If the query contains keywords like "talk to human", "manager", "complaint".
 - If the query is detected as out-of-domain or overly complex (e.g., "refund my money for order #123").
- Answer generation criteria: Standard factual queries about the product/document (e.g., "What are the support hours?").

5. HITL Design

- When escalation is triggered: The router outputs [requires_human=True].
- What happens after escalation: The graph execution goes to [hitl_escalation_node]. In a production app, this node sends a payload to a ticketing system (e.g., Zendesk) or a human dashboard and pauses execution ([interrupt_before=["hitl_escalation_node"]]).
- Integration: The human reviews the [state["question"]] and [state["context"]] (if any), provides the answer manually, and resumes the graph execution by injecting the human's response into [state["answer"]].

6. API / Interface Design

- Input Format: [{"question": "How do I reset my password?"}]
- Output Format:

[See diagram]

- Interaction Flow: User POSTs to [/ask]. The backend initializes LangGraph state, invokes the graph, and returns the resulting JSON.

7. Error Handling

- Missing Data: If the PDF is empty or missing, the system catches [FileNotFoundError] and returns a clear error message.
- No Relevant Chunks Found: If ChromaDB similarity scores fall below a threshold, the system triggers the HITL logic automatically (low confidence).
- LLM Failure: Implement retry logic (e.g., [Tenacity] or LangChain's built-in fallbacks) to handle API timeouts or rate limits gracefully.